Apples, Oranges, and Signatures
Pitfalls and Methodology in ML-DSA Benchmarking

Sebastien Riou[1], Jong-Yeon Park[2], Liga Anwar[2], Axel Poschmann[1], and Michael Hutter[1,2]

[1] PQShield Inc.,
Prama House, 267 Banbury Rd, Oxford, UK
{sebastien.riou,axel.poschmann}@pqshield.com
[2] University of the Bundeswehr Munich,
Werner-Heisenberg-Weg 39, 85579 Neubiberg, Germany
{jongyoun.park,liga.anwar,michael.hutter}@unibw.de

Abstract. Cryptographic migration, particularly in the post-quantum setting, poses significant practical challenges and requires reliable performance data to support sound engineering decisions. For ML-DSA, however, existing benchmarking practices often produce misleading or non-comparable results, complicating migration and cryptographic agility efforts. This paper analyzes common pitfalls in benchmarking ML-DSA signature operations, including subtle inconsistencies when comparing security levels. We show that execution-time variability of the ML-DSA signing algorithm—an inherent property due to rejection sampling and other data-dependent components—makes commonly used straightforward metrics, e.g., *min*/*average*/*max*, unsuitable for migration planning. To address this gap, we propose a robust benchmarking methodology based on standardized input data sets and clearly qualified reporting metrics. The proposed approach enables fair comparison across hardware and software implementations and supports designers of real-time systems to assess the worst-case execution time.

Keywords: Post-Quantum Cryptography (PQC) · ML-DSA· Worst-case execution time (WCET) · Cryptographic Migration · ASIL · Benchmarking Methodology · Cryptographic Agility.

1 Introduction

Benchmarking cryptographic primitives has a long history. Early efforts such as eBACS and SUPERCOP established unified benchmarking harnesses for classical cryptography [5], enabling cross-platform comparisons under controlled conditions. Similar goals motivated post-quantum benchmarking frameworks and reference platforms, notably the Open Quantum Safe project [20] and the PQM4 framework [10], which support systematic evaluation of post-quantum algorithms on embedded platforms. In practice, the resulting performance figures often guide product selection, architectural design, and cryptographic migration decisions, and these initiatives have strongly shaped current benchmarking practice.

Building on this foundation, several recent works have highlighted methodological limitations in existing benchmarking practices for post-quantum cryptography, particularly on constrained platforms. Howe et al. [8] evaluate lattice-based post-quantum signatures on ARM Cortex-M7 and show the strong impact of implementation choices, platform characteristics, and input sizes on measured performance, while also illustrating the difficulty of comparing results without a standardized methodology. Kris Gaj [6] explicitly frames post-quantum benchmarking as a methodological challenge, identifying sources of misleading comparisons such as variable-time behavior, differences in side-channel protection, ambiguous metrics, and limited reproducibility. A complementary perspective is provided by Kaps et al. [11], who propose a framework for fair and efficient benchmarking of lightweight cryptographic hardware. Although not focused on post-quantum schemes, their emphasis on clear measurement boundaries, standardized inputs, and avoiding cherry-picked operating points is directly applicable.

In this paper, we address a gap that remains largely unfilled in post-quantum signature benchmarking: the lack of a scheme-specific and rigorously defined methodology for benchmarking NIST FIPS 204 ML-DSA signature generation [15], arguably the most important quantum-safe signature algorithm. While prior frameworks and studies provide valuable performance data and general guidance, even after a decade of scrutiny they do not resolve the concrete ambiguities that arise when benchmarking ML-DSA, which combines multiple standardized interfaces, operation modes, variable-size inputs, and rejection-based execution. As a result, published benchmarks are often difficult to interpret or compare in a fair and migration-relevant manner. To the best of our knowledge, this is the first work to systematically analyze these ML-DSA-specific issues and to propose a concrete methodology enabling fair, reproducible, and meaningful performance comparison of ML-DSA signature implementations in post-quantum cryptographic migration contexts.

Contributions. This work makes the following contributions:

1. An analysis of algorithm-intrinsic sources of execution-time variability in ML-DSA, explaining why commonly reported average and worst-case performance figures can be ambiguous or misleading.
2. A systematic identification and formalization of benchmarking pitfalls specific to ML-DSA.
3. A novel benchmarking methodology specifically crafted for ML-DSA sign operation.
4. Reproducible artifacts including the first publicly verifiable WCET [22] benchmarking results for ML-DSA signing.

The remainder of this paper is organized as follows. Section 2 introduces ML-DSA, benchmarking concepts, WCET and ASIL context, and defines the scope of this work. Section 3 examines algorithmic sources of execution-time variability, while Section 4 discusses benchmarking pitfalls specific to ML-DSA signature generation. Section 5 presents a robust benchmarking methodology and

Table 1. ASIL level target FIT rates

ASIL Level	PMHF
ASIL A	None
ASIL B	≤ 100 FIT
ASIL C	≤ 100 FIT
ASIL D	≤ 10 FIT

Table 2. Maximum signing rate vs. ASIL level and timeout probability (signatures/s)

ASIL Level	Failure Probability		
	2^{-32}	2^{-48}	2^{-64}
ASIL B and C	0.0012	78.2	5.12 M
ASIL D	0.0001	7.8	512.40 K

reporting guidelines. Section 6 reports benchmarking results obtained using this methodology, and Section 7 concludes the paper, outlining what is left as future work.

2 Background and scope

We begin by outlining the industrial and real-time system context relevant to benchmarking, before introducing ML-DSA-specific considerations.

Benchmarking, WCET, and ASIL. The automotive industry adopts the ISO 26262 standard [1] and its Automotive Safety Integrity Levels (ASIL) to quantify system reliability, ranging from ASIL A (least stringent) to ASIL D (most stringent). Reliability is expressed using the Probabilistic Metric for Random Hardware Failure (PMHF), measured in Failures in Time (FIT), which estimates the number of failures per billion operating hours. The PMHF budget applies to the entire electronic control unit, and individual components are therefore allocated only a fraction of the FIT numbers given in Table 1; for example, Microchip reports that an MCU typically consumes about 10 % of the total FIT budget [13].

In real-time systems with a Worst-Case Execution Time (WCET) [22] constraint, an ML-DSA signing operation that exceeds its time budget constitutes a failure. Although such timeouts arise from the algorithm's statistical behavior and are therefore systematic rather than random hardware faults, they still contribute to the PMHF. This contribution depends on both the signing rate over the product lifetime and the probability of a timeout. Table 2 summarizes the maximum admissible signing rates for different ASIL levels, assuming that 1 % of the total FIT budget is allocated to ML-DSA signing. What is an acceptable timeout probability depends very much on the targeted application. Nevertheless it is clear that a failure probability of 2^{-32} is already too high for most applications at ASIL B or above, while probabilities around 2^{-48} should be adequate for most use cases at ASIL B and C. High-intensity use cases such as V2X at ASIL D may require failure probability below 2^{-48}. This highlights the importance of benchmarking rare execution-time cases and explicitly reporting both their execution times and associated probabilities of occurrence.

ML-DSA. ML-DSA, standardized by NIST [15], is a post-quantum digital signature scheme that balances security, performance, and implementation complexity. The specification includes three parameter sets corresponding to distinct NIST security levels:

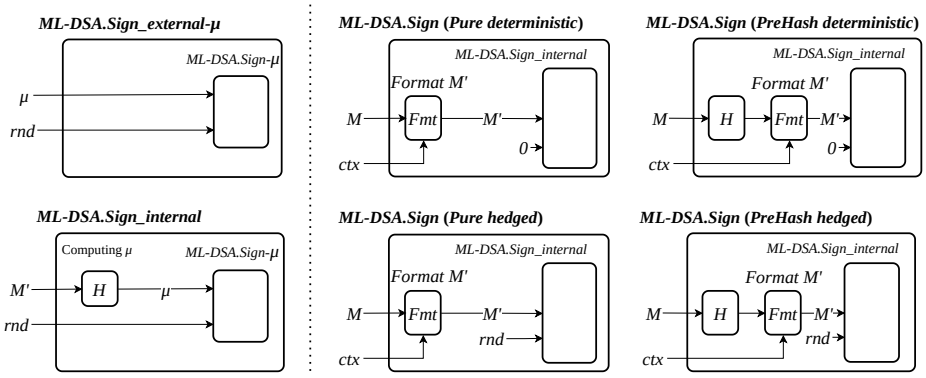

Fig. 1. Diagrams distinguishing ML-DSA.Sign variants.

- ML-DSA-44: NIST security level 2 (comparable to SHA-256)
- ML-DSA-65: NIST security level 3 (comparable to AES-192)
- ML-DSA-87: NIST security level 5 (comparable to AES-256)

Figure 1 illustrates the variants of ML-DSA.Sign specified in [15] and ML-DSA.Sign_external-μ. The left part describes building blocks while the right part describes the current *External interfaces*, i.e., standardized operations suitable for usage at application level. We introduced the ML-DSA.Sign-μ building block to show the relation between ML-DSA.Sign_external-μ and ML-DSA.Sign_internal. This highlights that ML-DSA.Sign-μ is the core operation for all variants.

The four variants of external interfaces affect only how the formatted message M' and how *rnd* are constructed. In *Pure* interface, the message M is formatted directly, whereas in *Pre-hash* interface (HashML-DSA) it is first hashed; apart from this step, both interfaces follow the same control flow and invoke the same internal signing algorithm. Both *Pure* and *Pre-hash* exist in *deterministic* and in *hedged* variants. In *deterministic* variant, the value of *rnd* is fixed to all zeros whereas in *hedged* variant it is randomized. Together, Figure 1 and Algorithm 1 show that the external interface is responsible only for constructing μ—including optional pre-hashing, hash-function selection, and incorporation of *ctx* (an application-defined context string) —while all security-critical operations are performed by a single common internal algorithm [15].

3 Sources of execution-time variability in ML-DSA

We first analyze the dominant source of execution-time variability in ML-DSA signature generation, followed by a discussion of other variable-time components.

Rejection sampling and signature repetitions. Independently of the selected variant and signing interface, ML-DSA signature generation exhibits inherently variable execution time, even when all input sizes are fixed. This variability stems from the algorithm's structure, which repeatedly generates candidate signatures until validity conditions are met, causing the number of signing-loop

Algorithm 1 ML-DSA.Sign_internal(sk, M', rnd)

Input: private key sk, formatted message $M' \in \{0,1\}^*$, $rnd \in \mathbb{B}^{32}$
Output: signature σ

$(\rho, K, tr, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0) \leftarrow \mathsf{skDecode}(sk)$
$\hat{\mathbf{s}}_1 \leftarrow \mathsf{NTT}(\mathbf{s}_1)$, $\hat{\mathbf{s}}_2 \leftarrow \mathsf{NTT}(\mathbf{s}_2)$, $\hat{\mathbf{t}}_0 \leftarrow \mathsf{NTT}(\mathbf{t}_0)$
$\hat{\mathbf{A}} \leftarrow \mathsf{ExpandA}(\rho)$ ▷ $\mathbf{A}$ generated/stored in NTT form
$\mu \leftarrow H(\mathsf{BytesToBits}(tr) \parallel M', 64)$
$\rho'' \leftarrow H(K \parallel rnd \parallel \mu, 64)$, $\kappa \leftarrow 0$, $(\mathbf{z}, \mathbf{h}) \leftarrow \perp$ ▷ random seed and counter
while $(\mathbf{z}, \mathbf{h}) = \perp$ **do** ▷ rejection sampling loop
  $\mathbf{y} \in R_q^\ell \leftarrow \mathsf{ExpandMask}(\rho'', \kappa)$
  $\mathbf{w} \leftarrow \mathsf{NTT}^{-1}(\hat{\mathbf{A}} \circ \mathsf{NTT}(\mathbf{y}))$
  $\mathbf{w}_1 \leftarrow \mathsf{HighBits}(\mathbf{w})$ ▷ signer's commitment
  $\tilde{c} \leftarrow H(\mu \parallel \mathsf{w1Encode}(\mathbf{w}_1), \lambda/4)$
  $\hat{c} \leftarrow \mathsf{NTT}(\mathsf{SampleInBall}(\tilde{c}))$
  $\langle\langle c\mathbf{s}_1 \rangle\rangle \leftarrow \mathsf{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{s}}_1)$, $\langle\langle c\mathbf{s}_2 \rangle\rangle \leftarrow \mathsf{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{s}}_2)$
  $\mathbf{z} \leftarrow \mathbf{y} + \langle\langle c\mathbf{s}_1 \rangle\rangle$, $\mathbf{r}_0 \leftarrow \mathsf{LowBits}(\mathbf{w} - \langle\langle c\mathbf{s}_2 \rangle\rangle)$ ▷ signer's response
  if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ **or** $\|\mathbf{r}_0\|_\infty \geq \gamma_2 - \beta$ **then**
    $(\mathbf{z}, \mathbf{h}) \leftarrow \perp$ ▷ validity checks failed
  else
    $\langle\langle c\mathbf{t}_0 \rangle\rangle \leftarrow \mathsf{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{t}}_0)$
    $\mathbf{h} \leftarrow \mathsf{MakeHint}(-\langle\langle c\mathbf{t}_0 \rangle\rangle, \mathbf{w} - \langle\langle c\mathbf{s}_2 \rangle\rangle + \langle\langle c\mathbf{t}_0 \rangle\rangle)$
    if $\|\langle\langle c\mathbf{t}_0 \rangle\rangle\|_\infty \geq \gamma_2$ **or** $\#1(\mathbf{h}) > \omega$ **then**
      $(\mathbf{z}, \mathbf{h}) \leftarrow \perp$
    end if
  end if
  $\kappa \leftarrow \kappa + \ell$ ▷ increment counter
end while
return $\sigma \leftarrow \mathsf{sigEncode}(\tilde{c}, \mathbf{z} \bmod^{\pm} q, \mathbf{h})$

repetitions to vary across signatures. This rejection-driven variability poses a fundamental challenge for benchmarking. While few rejections are common, much higher counts are not exceptional, and even a few hundred measurements may show large execution-time fluctuations.

Widely used benchmarking infrastructures such as SUPERCOP/eBACS [5], the Open Quantum Safe tools in `liboqs` [20], and PQM4 [10] provide valuable performance measurements across cryptographic implementations. Since the early releases of SUPERCOP and the eBACS project, multiple timing measurements have been collected and quartile-based statistics have been reported to expose execution-time variability. More recently, stabilized quartiles were introduced for rejection-sampling timings [4] and subsequently adopted by SUPERCOP/eBACS. These statistics characterize the timing distribution and make variability visible. However, these frameworks provide general-purpose benchmarking infrastructures and reporting mechanisms. They do not define standardized benchmark data sets or a methodology for reproducible evaluation of rare execution-time events with documented occurrence probabilities, which are relevant for real-time and safety-critical analyses. This work addresses these complementary aspects for ML-DSA.

In fact, execution-time variability is intrinsic to the ML-DSA algorithm: its timing distribution is heavy-tailed and sensitive to input selection and repetition statistics. Applying standard benchmarking workflows without additional qualification can therefore yield misleading or non-comparable results, motivating the scheme-specific methodology developed in this work.

Other variable-time components. Beyond the rejection-driven variability of the signing loop, several additional components contribute to execution-time variation even when input sizes are fixed:

- *Public-key–dependent variability.* Some variable-time components depend only on the public key. In particular, `ExpandA` relies on rejection sampling in `RejNTTPoly`, introducing execution-time variability independent of the message or signature and present before message-dependent processing.
- *Message- and secret-key–dependent variability.* Other components exhibit data-dependent behavior influenced by the message, context, and secret key, including `HighBits`, `LowBits`, `SampleInBall`, and `MakeHint`. In addition, failed validity checks may cause early termination or additional processing, further increasing execution-time variability.
- *Signature-dependent variability.* Execution time may also depend on the signature itself. The `SigEncode` stage, particularly `HintBitPack`, may process a variable number of coefficients, causing variability even after the signing loop completes.
- *Implications for benchmarking.* These effects show that execution-time differences are not solely determined by signing-loop rejections. Benchmarking ML-DSA implementations must therefore account for both repetition-driven and data-dependent timing behavior.

4 Pitfalls in ML-DSA signature benchmarking

In this section, we discuss benchmarking pitfalls relevant to ML-DSA signature generation. We first examine general pitfalls applicable to cryptographic benchmarking (Pitfalls 1–3), and then focus on pitfalls that arise from ML-DSA-specific design choices (Pitfalls 4–14).

Pitfall 1: comparison of different key sizes. Like many standardized cryptographic schemes, ML-DSA is specified for multiple security levels, corresponding to the parameter sets ML-DSA-44, ML-DSA-65, and ML-DSA-87, which differ substantially in computational cost, memory footprint, and security strength. Consequently, their performance characteristics are not directly comparable.

A common benchmarking pitfall is to compare signature generation performance across different ML-DSA parameter sets without clearly stating the security level. Such comparisons are misleading, as apparent performance gains often stem from lower security parameters rather than superior implementations. Performance results must be reported separately for each ML-DSA parameter set, and comparisons are only valid when the same security level is used. This best practice is exemplified by [2], where results are consistently presented per NIST security level.

Pitfall 2: protected vs. unprotected implementations. In cryptographic benchmarking, the distinction between protected and unprotected implementations is often overlooked. Although not mandated by the specification, such protections are essential under many real-world threat models [9,12,18].

A common pitfall is to compare implementations with different levels of protection against side-channel attacks (SCA) and fault-injection attacks (FIA) without explicitly accounting for these differences. Countermeasures such as masking or hiding can introduce substantial overhead and may interact non-trivially with rejection sampling and randomness generation.

For meaningful evaluation, benchmarks should jointly report performance, footprint, and security claims against SCA and FIA. When multiple configurations exist, all metrics should be given for each considered configuration, rather than, for example, reporting performance of the *maximum-performance* configuration, footprint of the *minimum-footprint* configuration, and security claims of the *maximum-security* configuration.

Pitfall 3: measurement boundaries. Benchmarking results depend critically on how measurement boundaries are defined. For ML-DSA signature generation, these boundaries are not uniquely specified and vary across implementations, particularly with respect to input parsing, hashing or pre-hashing, randomness generation, signature encoding, and memory transfers.

Comparing performance figures obtained with different, often undocumented, boundaries is a common pitfall. Benchmarks may exclude preprocessing or randomness generation, or measure only the cryptographic core while omitting data transfers, substantially affecting reported performance even for the same ML-DSA parameter set. For fair and reproducible benchmarking, measurement boundaries must be clearly defined and applied consistently. Otherwise, reported results may reflect benchmarking scope rather than true implementation efficiency.

Pitfall 4: comparison across ML-DSA interfaces. A common benchmarking pitfall is to compare performance across these interfaces as if they were equivalent. Such comparisons are misleading: the *Internal* and *External-μ* interfaces omit computations mandatory for a standard-compliant signature, such as message formatting and hashing, and therefore cannot be meaningfully compared to the *Pure* or *Pre-hash* interfaces. Apparent speedups reflect reduced computational scope rather than superior implementation efficiency.

For fair benchmarking, it is essential to distinguish between application-level signature interfaces (*Pure* and *Pre-hash*) and internal building blocks (*Internal* and *External-μ*). Performance comparisons are only meaningful when the same ML-DSA interface is used and all required computations are included.

Pitfall 5: ambiguous input sizes. Benchmarking of digital signature schemes often varies only the message size, implicitly assuming all other inputs have fixed length. For ML-DSA, this assumption is incorrect and can lead to misleading results. Depending on the signing interface, ML-DSA exposes multiple variable-

Table 3. Inputs of the different ML-DSA signing interface as defined in FIPS 204.

Input	Size (Bytes)	Pure	Pre-hash	Internal	External μ
sk (secret key)	Fixed	✓	✓	✓	✓
ctx (context)	0–255	✓	✓	–	–
M (message)	Variable	✓	✓	–	–
$\mathbf{M}'$ (formatted msg)	Variable	–	–	✓	–
rnd (randomness)	32	–	–	✓	✓
$\boldsymbol{\mu}$ (msg representative)	64	–	–	–	✓

length inputs, including the context string, formatted message, and per-message randomness.

Table 3 summarizes the inputs required by the different ML-DSA signing interfaces. As shown, the *Pure*, *Pre-hash*, and *Internal* interfaces each include at least one variable-size input beyond the message itself. In particular, the *Internal* interface operates on a formatted message whose size depends on standard-defined preprocessing, and the context input may range from zero to several hundred bytes, directly affecting signing cost.

Ignoring these variable-size inputs, or silently fixing them to convenient values, can distort benchmarking results. Performance figures that qualify only the message length are therefore insufficient. For meaningful benchmarking, all variable-size inputs relevant to the selected ML-DSA interface must be explicitly specified and consistently controlled.

Pitfall 6: comparison of *hedged* and *deterministic* variants. The ML-DSA standard supports both *hedged* and *deterministic* variants of signature generation. In the *hedged* variant, the per-message randomness input *rnd* is filled with fresh random data, whereas in the *deterministic* variant it is fixed to a constant.

A common benchmarking pitfall is to compare *hedged* and *deterministic* measurements without clearly qualifying the variant and signing interface. For the *Internal* and *External-*μ interfaces, *rnd* is provided explicitly and processed identically by the signing algorithm in both variants; any performance difference between *hedged* and *deterministic* variants in this case stems from external randomness generation rather than the ML-DSA implementation itself. In contrast, for the *Pure* and *Pre-hash* interfaces, randomness generation is performed internally and may introduce non-negligible overhead, conflating randomness costs with the intrinsic signing cost.

For fair benchmarking, the selected variant must be explicitly stated and applied consistently. Performance comparisons are only meaningful when the same ML-DSA interface and the same variant—*hedged* or *deterministic*—are used.

Pitfall 7: assuming *deterministic* is always faster. It is often assumed that *deterministic* ML-DSA signing is faster than the *hedged* variant because it avoids fresh randomness, but this assumption does not generally hold. In software implementations, randomness generation may add overhead, whereas many hardware accelerators take *rnd* as an input even for the *Pure* and *Pre-*

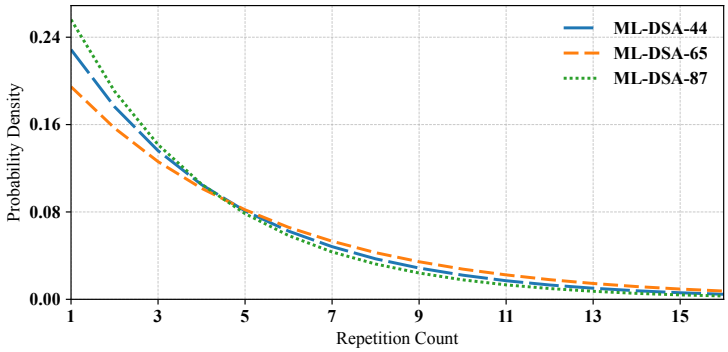

Fig. 2. Empirical distributions of ML-DSA signing-loop repetition counts over 20 million signatures for different parameter sets.[4]

hash interfaces, leaving signing time unchanged. In addition, rejection-sampling variability can dominate overall cost, eliminating any systematic advantage of *deterministic* signing.

Randomized signing can even improve performance in protected implementations by providing implicit masking against DPA [12]. As shown at the NIST Fourth PQC Standardization Conference [3], masked implementations using randomized signing achieve significant speedups (approximately 1.6×–1.7×) over *deterministic* designs. Consequently, assuming *deterministic* variants are always faster is a benchmarking pitfall.

Pitfall 8: ambiguous use of *average* and *maximum* execution time. Section 3 highlighted the intrinsic timing variability of ML-DSA sign operation. A common benchmarking practice is to execute the algorithm repeatedly on random inputs of fixed size and report statistics such as minimum, average, or maximum execution time. While suitable for mitigating environmental measurement noise, this approach is ill-suited when execution-time variability is heavy-tailed, like figure 2 illustrates. In the case of ML-DSA, it has several drawbacks:

- *Observed maxima are not true worst cases.* They reflect only the worst outcome encountered during measurement and may significantly underestimate the algorithmic worst case.
- *Results depend on sample size.* Benchmarks based on different numbers of executions are not directly comparable, as larger samples are more likely to expose rare high-rejection events.
- *Results are sensitive to input selection.* For some signing interfaces and variants, inputs can be chosen to statistically favor low rejection counts, yielding overly optimistic performance figures.

Without careful qualification, averages and maxima can therefore be misleading. For ML-DSA, execution-time variability must be treated as an explicit algorithmic property and incorporated into the benchmarking methodology, rather than averaged away as measurement noise.

Pitfall 9: misinterpreting maximum repetition bound. A natural question when benchmarking a variable-time algorithm is how to interpret its worst-case execution time. For ML-DSA signature generation, a standard-conforming implementation may perform up to 814 rejection iterations before aborting, as specified in FIPS 204 Appendix C [15]. This bound is a theoretical safety limit to guarantee termination, not a realistic execution scenario.

Treating this limit as a meaningful worst case is a serious benchmarking pitfall. The probability of reaching such an extreme number of rejections is vanishingly small—on the order of 2^{-256}. In practice, we observed a maximum of only 106 signing-loop iterations over 1.6 trillion ML-DSA-44 signatures, making the specification-defined abort bound grossly pessimistic and unsuitable for system sizing or performance comparison.

More generally, system-level guidelines caution against loop constructs that are theoretically bounded yet effectively non-terminating in practice. For example, the SEI CERT C Coding Standard (MSC21-C) stresses analyzable termination guarantees [19]. A similar issue arises in RSA key generation, where standards bound Miller–Rabin rounds but not the number of candidates tested [14], underscoring that probabilistic cryptographic procedures must be evaluated with real system constraints in mind.

Pitfall 10: computing average over skewed data. We analyze subsets of consecutive ML-DSA-87 signatures and compute the sum of repetitions for different sample sizes. Table 4 summarizes the results.

Table 4. Cumulative repetition variation for consecutive ML-DSA-87 signature subsets

Sample size	Min. sum	Max. sum	Theoretical
10	12	108	38.5
100	270	579	385.0
1,000	3,543	4,293	3,850.0
10,000	37,979	40,143	38,500.0

Even when benchmarking with 10,000 *hedged* signatures, two independent and well-intentioned benchmarks of the same implementation may report average execution times that differ by several percent. In the example above, one benchmark may observe an average corresponding to approximately 99 % of the theoretical value, while another may report 104 %. Put differently, results for a single implementation may vary by around 5 % solely due to statistical effects, despite using identical parameters and sample sizes.

Pitfall 11: extrapolating from repetition counts. One frequently proposed approach to address Pitfall 10 is to extrapolate execution time from a measured *time per repetition*. In this method, the signing time is measured for cases with one and with two repetitions, a nominal $Time_{repetition}$ is derived from the difference, and overall performance is estimated using the expected repetition count.

[4] See also Section 5.1 of https://datatracker.ietf.org/doc/draft-ietf-pquip-pqc-hsm-constrained/04.

Although this approach appears attractive and difficult to game, it is fundamentally flawed. In most implementations, $Time_{repetition}$ is itself variable. The signing loop may terminate at different points in Algorithm 1, each involving multiple conditional checks, and some implementations employ early-abort mechanisms that stop processing as soon as a failure is detected. As a result, there is no single, well-defined execution time for a single repetition.

Extrapolating overall signing performance from repetition counts and a nominal $Time_{repetition}$ therefore constitutes a benchmarking pitfall. Such extrapolation obscures intrinsic algorithmic variability and produces results that are neither robust nor comparable across implementations.

Pitfall 12: verifiable but incorrect signatures. In ML-DSA, a signature that was not produced by a correct signing execution may still be accepted by the verification algorithm. This does not indicate a flaw, but reflects a structural property of the scheme: verification checks only a subset of the conditions enforced during signing.

During signing, ML-DSA applies norm bounds and rejection sampling to maintain its security properties, whereas verification focuses only on algebraic consistency and selected public bounds, as dictated by the Fiat–Shamir-with-aborts construction. As a result, successful verification alone does not guarantee that a signature was generated by a compliant signer.

A concrete example is the rejection condition in line 19 of Algorithm 1,

$$\left[\left\|\langle\langle ct_0 \rangle\rangle\right\|_\infty \geq \gamma_2\right] \ \vee \ [\#1(h) > \omega] \tag{1}$$

which is enforced only during signing. Signatures violating this condition may still verify, even though they would never be produced by a correct implementation. Such violations are extremely rare—none were observed in over one billion signatures—making them unlikely to be detected by random sign–verify tests. Consequently, sign–verify consistency alone is insufficient to validate an ML-DSA implementation. Correctness testing must also ensure enforcement of signing-side constraints, such as norm bounds, rejection conditions, and randomness handling, to prevent implementations that appear correct while weakening the scheme's security guarantees.

Pitfall 13: unspecified *Pre-hash* function selection. In the *Pre-hash* interface of ML-DSA, the choice of the pre-hashing function is part of the signing interface and has a direct impact on performance. A common benchmarking pitfall is to report *Pre-hash* results without specifying the hash function used.

To ensure meaningful and reproducible benchmarking, results for the *Pre-hash* interface must explicitly state the hash function and relevant implementation details; otherwise, comparisons can be misleading.

Pitfall 14: pre-computations in key generation. A part of the sign operation can be precomputed as part of key generation function. It is therefore important to cover also the key generation in the benchmark otherwise implementations which adopt this strategy appear better than they really are.

5 Towards fair and reproducible ML-DSA Benchmarking

This section outlines the guiding principles and practical recommendations required for fair and reproducible benchmarking of ML-DSA implementations.

5.1 What *fair benchmarking* means

In the context of ML-DSA, we define *fair benchmarking* along two complementary dimensions:

1. *Fairness towards readers:* Reported performance figures must not mislead the reader into an overly optimistic interpretation. In particular, benchmark results should reflect realistic operational behavior rather than best-case or artifact-driven measurements.
2. *Fairness towards vendors (IP providers):* Benchmarking results should be robust against metric selection and configuration choices. In particular, an implementation with inferior or equal intrinsic performance should not be able to advertise better results by exploiting alternative metrics or benchmark setups, and repeating the benchmark on the same implementation should yield comparable results within expected statistical variation.

These objectives are partially conflicting, particularly when reporting *worst observed* execution times. Including higher rejection counts improves coverage of extreme behavior but increases the risk of unfair comparisons if competing benchmarks exclude such cases. This tension must therefore be managed through explicit reporting: benchmarks should state both the maximum observed number of signing-loop repetitions and the associated occurrence probability.

Table 5. Repetition counts for ML-DSA and occurrence probabilities.

Algorithm	**Occurrence Probability**		
	2^{-32}	2^{-48}	2^{-64}
ML-DSA-44	83	125	166
ML-DSA-65	102	153	204
ML-DSA-87	74	111	148

Table 5 summarizes repetition counts for selected target probabilities. A good benchmarking data set should reach the same occurrence probability for each key size, otherwise one may end up with a *worst observed* execution times higher for ML-DSA-44 than ML-DSA-65 for example. Similarly the average number of repetitions should match the theoretical average given in [15] Table 1:

- ML-DSA-44: 4.25 repetitions on average
- ML-DSA-65: 5.10 repetitions on average
- ML-DSA-87: 3.85 repetitions on average

Ideally, benchmarking data sets would include test vectors with occurrence probabilities of 2^{-48} or lower. Since generating such vectors is resource intensive, practical benchmarks may target higher probabilities, but the chosen probability must always be explicitly documented.

5.2 Handling the *hedged* variant

Constructing a fixed input data set with a prescribed average repetition count is not feasible for the *hedged* variant, due to the fresh randomness used for each signature. Its performance can instead be inferred from the *deterministic* variant, since the only algorithmic difference is the generation of 32 bytes of random data.

We propose measuring the *hedged* variant on random inputs of the same sizes and stopping once a zero-rejection signature is observed, estimating the randomness overhead as the timing difference between the *hedged* and *deterministic* variants. Although this does not capture all signing-loop variability, it avoids the error amplification inherent in repetition-based extrapolation. For implementations that do not support the *deterministic* variant, fair benchmarking requires exposing the number of signing-loop repetitions (Algorithm 2), at the cost of a resource-intensive search for suitable executions.

Algorithm 2 Benchmark procedure for *hedged*-only signing implementation

Input: a set of test vectors tvs
Output: minimum, average, and worst observed cycles

$remaining \leftarrow \mathsf{Repetitions}(tvs)$ ▷ List of repetitions occurring in tvs
$sk \leftarrow tvs.sk, \quad m \leftarrow tvs.messages[0], \quad ctx \leftarrow tvs.contexts[0]$
$sum \leftarrow 0, \quad cycles_{\min} \leftarrow \infty, \quad cycles_{wob} \leftarrow 0, \quad \mathrm{NP} \leftarrow |remaining|$
while $remaining$ **is not empty do**
  $(cycles, repetitions) \leftarrow \mathsf{Implementation.Sign}(sk, m, ctx)$ ▷ inputs are fixed
  if $repetitions \in remaining$ **then**
    remove $repetitions$ **from** $remaining$ ▷ avoid duplicate measurements
    $sum \leftarrow sum + cycles$
    $cycles_{\min} \leftarrow \min(cycles_{\min}, cycles), \quad cycles_{wob} \leftarrow \max(cycles_{wob}, cycles)$
  end if
end while
$cycles_{\mathrm{avg}} \leftarrow sum/\mathrm{NP}$ ▷ one measurement per one test vector
return $cycles_{\min}, cycles_{\mathrm{avg}}, cycles_{wob}$

5.3 Rare rejection path test vectors

Test vectors for rare signing-side rejection paths are necessary because these paths are infrequent yet performance-critical, making them easy to omit or misimplement. Developers may intentionally bypass such checks to improve apparent performance or inadvertently skip them due to misunderstandings or aggressive optimizations. Because these conditions are unlikely to be triggered by random inputs, standard benchmarking and validation procedures may fail to detect such deviations.

Without targeted test vectors, implementations that violate the signing specification may therefore appear faster while still passing verification-based tests, undermining fair performance evaluation. Alternatively, benchmark reports may document the results of dedicated functional test suites such as NIST's ACVP-Server [16] or Project Wycheproof [7].

5.4 Optimal message sizes

Optimal message size for the *internal* interface. Message size directly affects signature and verification performance. For the Internal signing interface, FIPS 204 defines the computation of the message representative as

$$\mu \leftarrow H\big(\text{BytesToBits}(\texttt{tr}) \,\|\, M'\big),$$

where $\texttt{tr} \in \mathbb{B}^{64}$ is defined in Algorithm 1 and

$$H(\text{str}, \ell) = \text{SHAKE-256}(\text{str}, 8\ell).$$

To minimize execution time, the goal is to ensure that μ is generated with a single invocation of the Keccak permutation. Let the input length to H be

$$|\texttt{tr}| + |M'| = 64 + |M'|$$

Accounting for the suffix overhead of SHAKE-256 (4 bits) and the `pad10*1` rule (2 bits), this corresponds to one byte of overhead in byte-oriented implementations. The maximum formatted message length is therefore

$$|M'| = \frac{1600 - 512}{8} - 64 - 1 = 71 \text{ bytes}.$$

Thus, messages up to 71 bytes yield optimal performance for Internal ML-DSA signing.

Optimal message size for the *pure* interface. For the Pure signing interface defines the formatted message as

$$M' \leftarrow \text{BytesToBits}\big(\text{IntegerToBytes}(0,1) \,\|\, \text{IntegerToBytes}(|\texttt{ctx}|,1) \,\|\, \texttt{ctx}\big) \,\|\, M$$

With a zero-length context, this gives $|M'| = 2 + |M|$. Using the same reasoning as above, the maximum message size yielding optimal performance is

$$|M| = 71 - 2 = 69 \text{ bytes}.$$

Optimal message size for the *pre-hash* interface. For the *pre-hash* interface (Algorithm 4 in [15]) applies a *pre-hash* function PH to the message. When using PH=SHAKE-128 and a zero-length context, the only dependency on message size arises from the input to SHAKE-128. Accounting for padding, the maximum message size processed with a single Keccak permutation is

$$|M| = \frac{1600 - 256}{8} = 168 \text{ bytes}.$$

Messages up to 168 bytes therefore achieve optimal performance for *pre-hash* ML-DSA signing.

For substantially larger messages (e.g., kilobytes or megabytes), reducing the number of Keccak invocations yields diminishing returns and has little impact on overall benchmark results.

5.5 Benchmarking ML-DSA.Sign: practical recommendations

Based on the pitfalls identified in this paper, we derive practical recommendations for benchmarking and reporting ML-DSA signature performance to support fair comparison, reproducibility, and informed deployment decisions.

Complete qualification of benchmarked operations. To avoid Pitfall 1, Pitfall 2, Pitfall 4, Pitfall 6 and Pitfall 13, benchmark reports should fully qualify the benchmarked operation and provide performance figures for all interfaces and variants supported by the product. At a minimum, the following parameters should be explicitly stated:

1. *Key size:* ML-DSA-44, ML-DSA-65, ML-DSA-87
2. *Signing interface: pure*, *pre-hash*, *internal*, *external-*μ
3. *Pre-hash algorithm:* e.g., SHAKE-256, SHA-512
4. *Variant: hedged*, *deterministic*

Explicitly describe the benchmarking boundaries. Precise reference to algorithms in [15] helps to avoid Pitfall 3.

Cover key expansion. Report figures for Algorithm 6 in [15] to avoid Pitfall 14.

Explicit reporting of input sizes. To avoid Pitfall 5, benchmark reports should always specify the input sizes used for benchmarking and, where applicable, select the optimal sizes for the corresponding signing interface:

- 69 bytes message, 0 byte context for *Pure*
- 168 bytes message, 0 byte context for *Pre-hash*
- 71 bytes formatted message for *Internal*
- For *External-*μ, message size is fixed by FIPS 204 via the message representative μ.

Dataset-based performance reporting. To avoid Pitfall 7, Pitfall 8, and Pitfall 9, performance figures should be derived from a standardized input data set rather than ad hoc measurements. Reports should include cycle counts over this data set with clearly labeled metrics such as *best observed*, *average*, and *worst observed*. The *worst observed* should have a documented probability of occurence. To avoid ambiguity, the benchmarking data set should be publicly accessible.

Constraints on standardized input data sets. For each key size, the standardized data set should target the *deterministic* variant and satisfy the following constraints:

- Include a best-case test vector with 0 rejections, including in `SampleInBall`.
- Exhibit an average repetition count matching the theoretical expectation (avoid Pitfall 10, Pitfall 11).
- Include a case with the highest number of rejections happening with the target probability.
- Include cases with rare rejection paths or mandate to pass an extensive functional test suite.

Table 6. Maximum repetition count for ML-DSA signing over 2 trillions signatures.

Algorithm	Occurrence Prob.			
	2^{-37}	2^{-39}	2^{-41}	2^{-43}
ML-DSA-44	96	-	106	-
ML-DSA-65	119	127	-	-
ML-DSA-87	86	-	-	100

Table 7. Maximum signing rate with 2^{-37} probability of timeout vs. ASIL level, 1 % of total FIT.

ASIL Level	Max. signatures/s
ASIL B and C	0.0382
ASIL D	0.0038

Implications for product comparison and cryptographic migration. Adhering to these recommendations enables meaningful and fair comparison between competing ML-DSA implementations across platforms and vendors. Reliable and reproducible benchmarking is also a prerequisite for cryptographic migration decisions, where performance data directly influences architectural choices, resource provisioning, and risk assessment during the transition to post-quantum cryptography.

6 Implementation Results

Test vectors to benchmark ML-DSA.Sign. Using an instrumented version of `mldsa-native` [17], we searched for executions exhibiting high signing-loop repetition counts in ML-DSA *Pure deterministic* signature generation. For this search, the message length was fixed to 69 bytes and the context length to 0 bytes. Table 6 summarizes the results; the column headers indicate the corresponding probabilities of occurrence.

As we found a case with a probability of occurrence of 2^{-37} for each key size, we use those cases to build our test vectors. Our search is far from reaching the target probability of 2^{-64} or even 2^{-48} so we will keep searching. Table 7 shows the number of signatures per second tolerable in ASIL levels for an application which would fail if the signature times out. We see that a probability of 2^{-37} is probably too high for ASIL B and C as it allows only 0.0382 signature per second. We intend to find lower probability test vectors in future works. For each key size, we add to the following test vectors (still with 69 bytes messages and 0 byte context):

- A zero-rejection case with zero rejection in `SampleInBall` (best case).
- Additional cases to reach an average repetition count matching the theoretical expectation for the target key size.

Finally, for each key size, we include a single zero-rejection test vector for two message lengths: 10 KB and 1 MB. All test vectors use a zero-length context. Each test vector is generated in four formats—C, Python, SystemVerilog, and JSON (following NIST's KAT format)—to facilitate reuse across software and hardware implementations. The complete set of test vectors is then bundled into a single archive and released publicly[5], together with the software used to generate them. The archive is named `mldsa-p37-m108-r96-1-1-119-1-1-86-1-1`, which encodes high-level information about the contained test vectors, i.e.,

[5] https://github.com/sebastien-riou/BTVG-MLDSA

- *p37*: probability of occurrence of worst case is 2^{-37}.
- *m108*: it contains 108 test vectors in total.
- *r96-1-1-119-1-1-86-1-1*: highest observed repetition counts per parameter set, namely 96 for ML-DSA-44, 119 for ML-DSA-65, and 86 for ML-DSA-87 on a 69-byte message, with a single repetition for both 10 KB and 1 MB messages in all cases.

We choose to not include cases with rare rejection paths. Instead we require the implementation to pass all test vectors from NIST's ACVP-Server [16] and from Project Wycheproof [7] test suites to maintain a clean separation between test vectors for functional validation and test vectors for performance benchmark. Benchmarking of the *hedged* variant is handled separately, as discussed in Section 5.2.

Benchmark on NUCLEO-U5A5 of ML-DSA.Sign *pure deterministic*. We use the test vectors `mldsa-p37-m108-r96-1-1-119-1-1-86-1-1` to benchmark software implementations on a NUCLEO-U5A5 development board [21]. This board integrates an STM32U5A5 microcontroller featuring an ARM Cortex-M33 CPU. All experiments are conducted at the maximum clock frequency of 160 MHz, with both instruction and data caches enabled. While operating the processor at a lower frequency would trivially yield smaller absolute cycle counts, such artificially reduced figures are less informative for practitioners seeking to determine whether a given implementation satisfies real-world timing constraints.

Table 8. Performance for a 69-byte message and zero-length context (in million cycles).

Implementation	Level	Operation	Minimum	Average[a]	Worst observed[b]
`pqcrystals-lowram`	44	key-exp	3.55	3.65	3.70
	44	sign	6.99	22.01	446.48
	87	key-exp	11.29	11.44	11.74
	87	sign	18.56	53.92	1061.51
`stm32pqc-lowram`	44	key-exp	2.25	2.27	2.28
	44	sign	4.20	14.61	308.69
	87	key-exp	6.46	6.47	6.50
	87	sign	9.95	32.99	691.94
`wolfssl-lowram`	44	key-exp	2.93	2.96	2.97
	44	sign	4.27	16.28	355.35
	87	key-exp	8.54	8.56	8.61
	87	sign	11.87	42.60	926.04

[a] Match long term average for `sign`. [b] Probability of occurrence is 2^{-37} for `sign`.

All software components are compiled using `arm-none-eabi-gcc` (`xPack GNU Arm Embedded GCC x86_64` version `14.2.1 20241119`), with the exception of the `stm32pqc` library, which is provided by STMicroelectronics in precompiled binary form (we evaluated the version v1.0.1). The measurement boundaries match Algorithm 6 in [15] for `key-exp`, *deterministic* variant of Algorithm 2 in

Table 9. Memory footprint for key generation, signing and verification (in KiB).

Implementation	Level	Stack	Heap	Static RAM	Read-only
`pqcrystals-lowram`	44	5.30	0.00	3.77	12.52
	87	8.30	0.00	7.30	12.21
`stm32pqc-lowram`	44	1.20	0.00	14.88	17.20
	87	1.20	0.00	26.41	17.04
`wolfssl-lowram`	44	1.98	28.01	20.32	25.94
	87	1.98	79.01	20.32	25.94

[15] for `sign` and Algorithm 3 in [15] for `verify`. All software involved is publicly released[2].

Table 8 presents the performance of ML-DSA-44 and ML-DSA-87 according to our methodology. It shows that even with its relatively high probability of occurrence, our worst case test vector for `sign` is about 20 times slower than the average execution time. Table 9 presents the memory footprint of ML-DSA-44 and ML-DSA-87. `Stack` and `Heap` contains the maximum size when calling successively `key-exp`, `sign` and `verify`. The `Static RAM` contains private and public keys in the native format of each implementation. We observe large variations from one library to another as various performance-memory trade-offs are possible in ML-DSA implementation.

None of the benchmarked implementations have claims about SCA or FIA. We leave the validation of those implementation with the functional validation suites as future work.

7 Conclusions

This paper examined the challenges of benchmarking ML-DSA signature generation and showed that conventional metrics can be misleading when methodological choices are left implicit. We identified 14 key pitfalls related to standardized interfaces, variable-size inputs, and rejection sampling that hinder fair performance comparison. We proposed a fair and reproducible benchmarking methodology based on publicly available input data sets, clearly qualified configurations, and reporting metrics, enabling fair comparison and more reliable performance data for post-quantum deployment and migration. We also presented the first ML-DSA signing benchmarks reporting worst-case execution time with associated occurrence probabilities, released the supporting software, and outlined future work extending these results to lower-probability test vectors and to HashML-DSA. While this work focuses on ML-DSA, the identified issues are not unique to this scheme. Other post-quantum signature and key encapsulation mechanisms similarly combine multiple standardized variants, variable-size inputs, and data-dependent execution paths. As a result, comparable benchmarking ambiguities are likely to arise unless scheme-specific methodologies are adopted. ML-DSA therefore serves as a concrete and timely case study illustrating a broader class of challenges in post-quantum cryptographic benchmarking.

[2] https://github.com/sebastien-riou/crypto-benchmark,
https://github.com/sebastien-riou/crypto-benchmark-stm32u5

Acknowledgments. This work has received funding from the EU's Horizon Europe research and innovation programme under grant agreement No. 101225722 (FORTRESS).

References

1. ISO 26262-1:2018 Road vehicles – Functional safety – Part 1: Vocabulary (2018), edition 2, 2018-12
2. Abbasi, M., Cardoso, F., Váz, P., Silva, J., Martins, P.: A Practical Performance Benchmark of Post-Quantum Cryptography Across Heterogeneous Computing Environments. Cryptography **9**(2), 32 (2025)
3. Azouaoui, M., Bronchain, O., Cassiers, G., et al.: Leveling Dilithium against Leakage, Revisited: Sensitivity Analysis and Improved Implementations. Presentation at the Fourth NIST Post-Quantum Cryptography Standardization Conference (Nov 2022), available at the NIST Computer Security Resource Center (CSRC)
4. Bernstein, D.J.: Robust Statistics for Rejection-Sampling Timings. https://cr.yp.to/papers/rsrst-20250727.pdf (Jul 2025), preprint
5. Bernstein, D.J., Lange, T.: eBACS: ECRYPT Benchmarking of Cryptographic Systems, http://bench.cr.yp.to
6. Gaj, K.: Challenges and Rewards of Implementing and Benchmarking Post-Quantum Cryptography in Hardware. In: Chen, D., Homayoun, H., Taskin, B. (eds.) Proceedings of the 2018 on Great Lakes Symposium on VLSI, GLSVLSI 2018, Chicago, IL, USA, May 23-25, 2018. pp. 359–364. ACM (2018)
7. Google Security Team: Project Wycheproof: Cryptographic Test Vectors. https://github.com/C2SP/wycheproof, accessed: 2026-01-28
8. Howe, J., Westerbaan, B.: Benchmarking and Analysing the NIST PQC Lattice-Based Signature Schemes Standards on the ARM Cortex M7. In: Mrabet, N.E., Feo, L.D., Duquesne, S. (eds.) Progress in Cryptology - AFRICACRYPT 2023 - 14th International Conference on Cryptology in Africa, Sousse, Tunisia, July 19-21, 2023, Proceedings. Lecture Notes in Computer Science, vol. 14064, pp. 442–462. Springer (2023)
9. Hutter, M., Tunstall, M.: Constant-Time Higher-Order Boolean-to-Arithmetic Masking. J. Cryptogr. Eng. **9**(2), 173–184 (2019), https://doi.org/10.1007/s13389-018-0191-z
10. Kannwischer, M.J., Rijneveld, J., Schwabe, P., Stoffelen, K.: pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4. Workshop Record of the Second PQC Standardization Conference (2019), https://cryptojedi.org/papers/#pqm4
11. Kaps, J., Diehl, W., Tempelmeier, M., Farahmand, F., Homsirikamol, E., Gaj, K.: A Comprehensive Framework for Fair and Efficient Benchmarking of Hardware Implementations of Lightweight Cryptography. IACR Cryptol. ePrint Arch. p. 1273 (2019), https://eprint.iacr.org/2019/1273
12. Kocher, P., Jaffe, J., Jun, B.: Differential Power Analysis. In: Advances in Cryptology–CRYPTO'99: 19th Annual International Cryptology Conference. pp. 388–397. Springer (1999)
13. Lassen, J.L.: Importance of the Failure Mode Effect and Diagnostics Analysis when Designing Safety Critical Applications. Microchip (2020), https://ww1.microchip.com/downloads/en/DeviceDoc/Failure-Mode-Effect-Diagnostics-Analysis-DS00003638A.pdf

14. National Institute of Standards and Technology: Digital Signature Standard (DSS). FIPS PUB 186-5 (2023), https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-5.pdf
15. National Institute of Standards and Technology: FIPS 204: Module-Lattice-Based Digital Signature Standard (Aug 2024), https://doi.org/10.6028/NIST.FIPS.204
16. National Institute of Standards and Technology (NIST): ACVP-Server: Automated Cryptographic Validation Protocol Server. https://github.com/usnistgov/ACVP-Server, accessed: 2026-01-28
17. PQ-Code-Package Project: mldsa-native: Native Implementation of ML-DSA. https://github.com/pq-code-package/mldsa-native, accessed: 2026-01-28
18. Sasdrich, P., Bilgin, B., Hutter, M., Marson, M.E.: Low-Latency Hardware Masking with Application to AES. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2020**(2), 300–326 (2020). https://doi.org/10.13154/TCHES.V2020.I2.300-326, https://doi.org/10.13154/tches.v2020.i2.300-326
19. SEI CERT: MSC21-C: Use Robust Loop Termination Conditions. SEI CERT C Coding Standard (2023), https://wiki.sei.cmu.edu/confluence/display/c/MSC21-C.+Use+robust+loop+termination+conditions
20. Stebila, D., Mosca, M.: Post-quantum Key Exchange for the Internet and the Open Quantum Safe Project. In: Avanzi, R., Heys, H.M. (eds.) Selected Areas in Cryptography - SAC 2016 - 23rd International Conference, St. John's, NL, Canada, August 10-12, 2016, Revised Selected Papers. Lecture Notes in Computer Science, vol. 10532, pp. 14–37. Springer (2016), https://doi.org/10.1007/978-3-319-69453-5_2
21. STMicroelectronics: NUCLEO-U5A5ZJ-Q Evaluation Board. https://www.st.com/en/evaluation-tools/nucleo-u5a5zj-q.html, accessed: 2026-01-28
22. Wilhelm, R., Engblom, J., Ermedahl, A., Holsti, N., Thesing, S., Whalley, D.B., Bernat, G., Ferdinand, C., Heckmann, R., Mitra, T., Mueller, F., Puaut, I., Puschner, P.P., Staschulat, J., Stenström, P.: The Worst-Case Execution-Time Problem - Overview of Methods and Survey of Tools. ACM Trans. Embed. Comput. Syst. **7**(3), 36:1–36:53 (2008), https://doi.org/10.1145/1347375.1347389